

Sys-MoDEL - MRG Collaborarion: Tidal Turbine Modelling in Modelica

Thomas Hogan¹, Carwyn Frost², and Kush Bubbar¹

¹System Model Development Engineering Lab, Department of Electrical and Computer
Engineering, University of New Brunswick

²Marine Research Group, Department of Civil Engineering, Queen's University Belfast

September 26, 2025

Abstract

The project investigated the modelling of tidal stream generators using the Modelica language. This work established the foundation for the tidal modelling component of the Modelica Ocean Engineering Toolbox. Given the exploratory nature of the project and the limited timeline, the aim was not to deliver a fully operational toolbox but to demonstrate a proof of concept showing that a Modelica-based framework is both **feasible** and **valuable**. The project concluded with experimental validation using medium-scale tank test data provided by QUB. Dr. Bubbar supervised the overall scope and direction of the project, while Dr. Frost contributed expertise in tidal stream modelling, testing, and tidal-specific aspects.

This report summarizes the work undertaken, provides documentation for future developers, and reflects on lessons learned. A brief review of the fundamental theory is followed by a description of the code implementation and an account of the attempted experimental validation. A link to the codebase in the GitHub repository can be found [here](#).

Contents

1	Introduction	1
2	Modelica	1
2.1	Functional Mock-Up Interface	1
3	Fundamental Theory	2
3.1	Rigid Body Kinematic Rotations	2
3.2	BEMT Theory	4
3.3	Ocean Engineering Toolbox Theory	8
4	Software	9
4.1	MATLAB BEMT Algorithm	11
4.1.1	Debugging BEMT Algorithm	12
4.1.2	Simulink BEMT Algorithm	13
4.1.3	Validation Scripts	14
4.2	Modelica Models	15
5	Debugging and Validation	18
5.1	Debugging	18
5.2	Validation	24
6	Conclusions and Future Work	25
6.1	Future Work	25
6.2	Recommended Literature	26

List of Figures

1	Wind Turbine Coordinate System [1]	3
2	Directory to the Tidal Code in the Ocean Engineering Toolbox GitHub Repository	9
3	Directory to the Debugging Code in the Ocean Engineering Toolbox GitHub Repository . . .	12
4	MATLAB Environment - Debugging Algorithm	12
5	Directory to the Simulink Code in the Ocean Engineering Toolbox GitHub Repository	13
6	MATLAB Environment - Simulink Algorithm	13
7	MATLAB Environment - Simulink Algorithm System Block	14
8	MATLAB Environment - Simulink Algorithm FMU	14
9	Directory to the Validation Code in the Ocean Engineering Toolbox GitHub Repository . . .	14
10	Directory to the Modelica Code in the Ocean Engineering Toolbox GitHub Repository	15
11	Directory to the Tidal Portion in the Ocean Engineering Toolbox Codebase	15
12	Sample Modelica FMU Configuration	16
13	Sample Modelica FMU Configuration	17
14	Blade Positions - Debugging Visualization	18
15	Incoming Wind - Debugging Visualization	19
16	Relative Velocity - Debugging Visualization	20
17	Angle of Attack - Debugging Visualization	21
18	Induced Velocity - Debugging Visualization	22
19	Aero/Hydrodynamic Loads - Debugging Visualization	23
20	BEMT Validation with Medium-Scale Towing Tank Experimental Data	24

Nomenclature

BEMT Blade element momentum theory

FMI Functional Mock-up Interface

FMU Functional Mock-up Unit

MRE Marine renewable energy

MSL Modelica Standard Library

OET Ocean Engineering Toolbox

OOP Object-Oriented Programming

1 Introduction

Several open-source and commercial codes implement sophisticated BEMT algorithms, including AeroDyn [2], Ashes [3], OrcaFlex [4], and MOST [5]. Many of these algorithms are derived from AeroDyn, which is by far the most mature tool. AeroDyn serves as the aeroelastic module within OpenFAST, alongside other integrated modules such as MoorDyn (mooring), TurbSim and InflowWind (wind), HydroDyn (wave hydrodynamics), ElastoDyn (tower, nacelle, rotor), ServoDyn (control and power generation), SubDyn (substructure) and SeaState (wave environment). Predominantly, the literature considers BEMT in the context of wind turbine applications rather than tidal. Accordingly, the present work describes the aerodynamic case, although the same formulation can be seamlessly applied to hydrodynamic loads.

The advantages of a Modelica-based approach lie in its capacity to model multi-physics systems (i.e., resource-to-wire modelling) and the flexibility to modify and interchange subsystem models seamlessly. The former supports a more accurate representation of the generator and control systems, while the latter facilitates model customization. However, a key limitation is the inability to represent blade elasticity, which should be addressed in future work.

Section 2 provides a brief overview of the Modelica language, Section 3 reviews the fundamentals of BEMT, Section 4 describes the software implementation, and Section 5 presents the attempted experimental validation.

2 Modelica

Modelica™ is a symbolic, equation-based, object-oriented modelling language designed to simulate cyber-physical systems, offering key advantages relative to traditional modelling software, such as [6]:

- **Equation-based modelling:** The Modelica™ language utilizes a declarative equation-based approach rather than explicit variable assignments, allowing systems to be described without imposing predefined causal relationships (acausal programming) and includes standard components that can be easily modified and reused. Multi-physics component models are formulated to directly represent the underlying physical system, eliminating the need for signal-based abstractions and facilitating integration across engineering domains.
- **Open-source standard library:** The Modelica Standard Library (MSL) is an open-source package maintained by the Modelica Association™ that contains standard modelling components in the mechanical, electrical, magnetic, thermal, and fluid domains. This work leverages the Modelica™ multibody systems (MBS) library (a 3D mechanical system package in the MSL). Being fully open-source, MSL components can be easily adapted by the user to suit their bespoke needs, which has made this work possible.
- **Interoperability:** Modelica™ software supports the standardized Functional Mock-up Interface (FMI), enabling interfacing with external cyber-physical models via co-simulation and model-exchange. Additionally, dedicated APIs facilitate scripting in Python, MATLAB, and Julia for enhanced computing and post-processing capabilities.

The simulations in this work are developed, compiled, and executed within the open-source OpenModelica (OMEdit) environment [7]. Two recommended sources for learning the Modelica language are [8] (for high-level examples) and [6] (for an in-depth description).

2.1 Functional Mock-Up Interface

The Functional Mock-up Interface (FMI) enables the export of pre-compiled dynamic models, called FMUs, which can be exchanged between different tools and simulated. FMI is supported by over 250 software tools. There are two types of FMUs: Model Exchange (ME) and Co-Simulation (CS).

The FMI standard has evolved through multiple versions. The original version, FMI 1.0, was followed by FMI 2.0, which introduced several enhancements and broader tool support. The most recent update, FMI 3.0, further improves compatibility and functionality across simulation environments.

1. Model Exchange (ME)

- Analogous to vector graphics (e.g., .pdf, .svg): functions provide access to the underlying mathematical model.
- The numerical solver is supplied by the importing tool.
- The FMU provides functions to set inputs/outputs and compute state derivatives.
- The importing tool determines the time steps and computes the next state.
- Best suited for simple models and allows greater flexibility.

2. Co-Simulation (CS)

- Analogous to bitmap graphics (e.g., .jpeg, .png): rendering (simulation) is controlled by the exporting tool.
- The numerical solver is embedded in the FMU.
- The importing tool sets inputs, instructs the FMU to advance, and reads outputs.
- Best suited for complex models.

3 Fundamental Theory

The fundamental BEMT theory is presented here, based primarily on the seminal work of Hansen [1], from which many subsequent formulations are derived. Branlard [9] provides additional detail and includes a partial code implementation. Concise summaries can also be found in [3] and [10]. Subsection 3.1 offers a brief overview of complex rotations and common turbine coordinate frames, while Subsection 3.2 presents the BEMT algorithm.

3.1 Rigid Body Kinematic Rotations

The rotations about the x, y, and z axes can be represented as simple rotations with $\mathbf{R}_x$ (1), $\mathbf{R}_y$ (2), and $\mathbf{R}_z$ (3) respectively. The rotation sequence is represented with Cardan angles in the intrinsic $z-y-x$ sequence as shown in 4.

$$\mathbf{R}_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \quad (1)$$

$$\mathbf{R}_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \quad (2)$$

$$\mathbf{R}_z(\gamma) = \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3)$$

$$\mathbf{R}(\alpha, \beta, \gamma) = \mathbf{R}_z(\gamma)\mathbf{R}_y(\beta)\mathbf{R}_x(\alpha) \quad (4)$$

Caution

By default, OpenModelica represents complex rotations utilizing Cardan angles in the intrinsic $x-y-z$ sequence. Therefore, the rotation sequence in OpenModelica must be updated from 1, 2, 3 to 3, 2, 1 to align with the convention defined in [1]. This returns the same rotation matrix; however, the resultant transformation from the *AbsoluteSensor* component returns a different quantity. It appears that if the manual rotation matrix is transposed, the same quantity can be achieved with both tools. This could arise from a difference in passive vs active rotations, but that is speculation.

The coordinate system description follows that defined in [1], which introduces four coordinate systems. Coordinate system 1 (the inertial system) is located at the base of the tower or platform; coordinate system 2 (nacelle) is non-rotating and positioned in the nacelle; coordinate system 3 (rotor shaft) is fixed to the rotating shaft; and coordinate system 4 is aligned with one of the blades [1]. A right-handed convention is used, with the z -axis defined along the rotor shaft in the direction of the incoming wind, and the x -axis pointing vertically upwards. An example of this turbine coordinate system is shown in Figure 1.

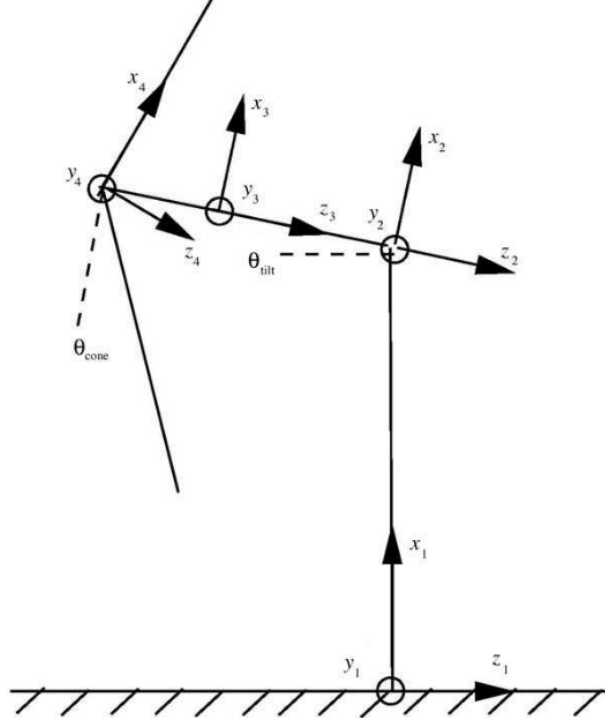


Figure 1: Wind Turbine Coordinate System [1]

Warning

This definition of the system is internally consistent; however, it is not aligned with the multibody dynamic and hydrodynamic formulations within the OET. The present work adhered to the original definition, but it must be modified to ensure consistency with the OET. In the OET, the global coordinate system (GCS) is defined at the undisturbed water level, with the positive x -axis oriented in the direction of wave propagation and the positive z -axis pointing vertically upwards [11].

Systems 1 and 2 are identical except for the location of their origins. System 2 is rotated about the x -axis by θ_{yaw} , about the y -axis by θ_{tilt} , and remains unrotated about the z -axis, as shown in

$$\vec{r}_2 = \mathbf{R}_{12}\vec{r}_1, \quad (5)$$

where $\vec{r}_1 = \{x_1, y_1, z_1\}$. System 2 is rotated about the z -axis by θ_{wing} to be transformed into system 3.

$$\vec{r}_3 = \mathbf{R}_{23}\vec{r}_2 \quad (6)$$

System 3 is rotated by θ_{cone} about the y -axis to system 4.

$$\vec{r}_4 = \mathbf{R}_{34}\vec{r}_3, \quad (7)$$

The selection of coordinate frames may differ slightly for tidal applications, but the overall formulation should remain similar. In particular, systems 1 and 3 must interface with the Modelica model. Unlike the fixed ground considered in this derivation, the floating platform (system 1) undergoes motion, and its kinematics must be provided by the hydrodynamic and multibody model to the BEMT algorithm. Similarly, system 3 corresponds to the rotor frame, where the rotor speed is obtained from the Modelica model and where the aero/hydrodynamic loads are applied.

3.2 BEMT Theory

BEMT is used to compute the aero/hydrodynamic loads on a rotor by combining blade element and momentum theories. The steady BEMT algorithm is often sufficient to obtain reasonable estimates of annual energy production; however, the flow experienced by the rotor is inherently unsteady due to evolving wakes, wind shear, and tower shadow [10]. To capture these unsteady effects, an unsteady BEMT algorithm is required. The steady algorithm assumes quasi-static conditions, whereas the unsteady algorithm incorporates the dynamics from the previous time step to compute the loads at the current step. The unsteady algorithm uses time as the relaxation instead of an iteration; however, this requires a sufficiently small step size to be reasonable.

In this work, where instantaneous dynamics must be considered for accurate real-time control, unsteady effects should be explicitly accounted for. Consequently, only the unsteady algorithm is described here. Since it is formulated as an extension of the steady theory, much of the underlying approach remains the same. The algorithm and corresponding equations (primarily in discrete form; however, subscripts will be dropped whenever it is clear) will be shown below.

Step 0: Initialization

The blade must be discretized and the parameters defined at each element (i.e. chord, radius, pitch) as well as other turbine parameters (i.e. hub parameters, airfoil, etc). The initial positions and velocities of the blade elements must also be defined. The blade position is particularly important, as the values are nonzero. The azimuthal positions of the blades are evenly distributed around the rotor, with the spacing determined by the number of blades.

Step 1: Incoming Wind

The incoming wind is computed with a power law formulation

$$V0(z) = V0(H) * (\frac{z}{\beta H})^\alpha, \quad (8)$$

where H is the nominal depth where the inflow is defined, z is the instantaneous blade element depth, $V0$ is the wind/flow speed, β is the bed roughness coefficient (specific to tidal, not applicable to wind), and α is the power law coefficient ($1/5-1/7$).

Error

The current work assumes a nominal hub depth and defines the wind/flow speeds at the blade elements relative to that fixed point using the x-coordinate of the blade element radius resolved in the inertial frame, added to the nominal hub depth ($z = H + Rbe_x^1$). By assuming the wind is always defined at the hub and only using the blade element radius and nominal hub depth, this neglects the motion of the platform and any motion between the platform and the hub. To account for these effects, the depth of the blade element (which is currently assumed to be the blade element radius and nominal hub depth) should instead be the actual depth of the blade element. The distance between the blade element and the hub is fixed, so that portion can remain unchanged, but instead of adding it to the nominal hub depth, it should be added to the actual hub depth. The actual hub depth (which is known) should be resolved in system 1 and added to the blade element depth in system 1.

Caution

Code for a simple tower shadow model has been included to account for the presence of a tower [1]; however, it does not work.

Step 2: Relative Velocity

The relative velocity of the blade element ($\vec{V}_{rel,i}$) is obtained as the sum of the incoming wind ($\vec{V}_{0,yz,i}$), the velocity due to blade deformation ($\vec{V}_{elas,i}$), the rotational velocity ($\vec{V}_{rot,i}$), and the induced velocity from the **previous** time step ($\vec{W}_{i-1}$). The superscript indicates the coordinate system in which it is defined, and the subscript corresponds to the time step.

$$\vec{V}_{rel,i}^4 = \vec{V}_{0,i}^4 + \vec{V}_{elas,i}^4 + \vec{V}_{rot,i}^4 + \vec{W}_{i-1}^4 \quad (9)$$

$$\begin{bmatrix} V_{rel,x}^4 \\ V_{rel,y}^4 \\ V_{rel,z}^4 \end{bmatrix} = \begin{bmatrix} 0 \\ V0_y^4 \\ V0_z^4 \end{bmatrix} + \begin{bmatrix} 0 \\ -\omega R_{be} \cos \theta_{cone} \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ W_y^4 \\ W_z^4 \end{bmatrix} \quad (10)$$

Where ω is the angular velocity and R_{be} is the radius of the blade element.

Note

The current formulation is LTI, meaning all bodies are assumed to be rigid and neglect deformation ($\vec{V}_{elas} = \vec{0}$).

Step 3: Angle of Attack

- (a) Compute the flow angle (ϕ).

$$\phi = \arctan\left(\frac{V_{rel,z}^4}{V_{rel,y}^4}\right) \quad (11)$$

- (b) Compute the angle of attack (α) from the twist (β) and pitch (θ_p)

$$\alpha = \phi - (\beta + \theta_p) \quad (12)$$

Warning

This work alternates between radians and degrees for visualization and computation; future work should ensure consistency.

- (c) Reynold's number (Re) from the chordlength (c) and the kinematic viscosity (ν)

$$Re = \frac{\|\vec{V}_{rel}^4\| c}{\nu} \quad (13)$$

Note

Reynolds' number is not directly used for any computation, but as a warning when computing the lift and drag coefficients. The lift and drag coefficients are functions of Reynolds' number and the angle of attack; however, in the regimes where tidal turbines operate, these coefficients are often independent of Reynolds' number ($Re > 10^5$).

Step 4: Tip, Hub, and Thrust Corrections

- (a) A correction is added for the computation of the new quasi-static induced velocity to account for the loss of lift at the tip and hub due to the finite number of blades (formulation assumes an infinite number of blades [3]. This is often performed with the Prandtl (F_P) tip (F_{tip}) and hub (F_{hub}) corrections.

$$F_P = F_{tip} \cdot F_{hub} \quad (14)$$

$$F_{tip} = \frac{2}{\pi} \cos^{-1} \left[\exp \left(\frac{-N_B(R_r - R_b)}{2R_b \sin \phi} \right) \right] \quad (15)$$

$$F_{hub} = \frac{2}{\pi} \cos^{-1} \left[\exp \left(\frac{-N_B(R_b - R_h)}{2R_b \sin \phi} \right) \right] \quad (16)$$

R_r is the rotor radius, R_b is the blade element radius, and R_h is the hub radius.

- (b) Glauert's correction is applied in cases with heavily loaded rotors (i.e., high values of the axial induction factor); it provides an empirical relationship between the thrust coefficient and the axial induction factor.

$$f_g = \begin{cases} 1, & a \leq a_c \\ a_c \left(2 - \frac{a}{a_c} \right), & a > a_c \end{cases} \quad (17)$$

a_c is the critical induction factor. [1] recommends a value of 0.2; however, [3] notes that this limit has been abandoned and instead recommends a value of 1/3.

Step 5: Lift and Drag Coefficients

- (a) Determine the static drag (CD_{st}) and lift (CL_{st}) coefficients for the airfoil. Nominally, this is a 1D interpolation given α , but it can also involve using Re as the second interpolating variable (discussed above).
- (b) The effects of changing the angle of attack do not occur instantaneously but rather with a delay that depends on the chord length and the relative velocity within the blade's boundary layer (which varies depending on whether the flow is attached or partially separated) [1]. To account for this, a dynamic stall model can be introduced, allowing the lift coefficient (C_L^{dy}) to be adjusted, particularly in cases of trailing-edge stall.

$$CL_{dy}(\alpha) = fCL_{inv}(\alpha) + (1 - f)CL_{fs}(\alpha), \quad (18)$$

where f is the separation function, CL_{inv} is the lift coefficient for inviscid flow without flow separation, and CL_{fs} is the lift coefficient for fully separated flow [3], [12].

C_L^{inv} is obtained by extrapolating the linear region of the lift curve. The static separation function, which has a theoretical upper limit of 1, is then applied.

$$f_{st} = \min \left(2 \sqrt{\frac{CL_{st}}{CL_{st,inv}}}, 1 \right) \quad (19)$$

The separation function is found as a first-order response, which can be analytically integrated as follows:

$$f_i = f_{st,i} + f_{i-1} - f_{st,i} e^{-\frac{\Delta t}{\tau}} \quad (20)$$

The fully separated lift coefficient can then be found as

$$CL_{fs}(\alpha) = CL_{st}(\alpha) - \frac{f_{st} CL_{st,inv}}{1 - f_{st}} \quad (21)$$

Now, the quantities determined in (19), (20), and (21) can be used to solve for (18).

Step 6: Aero/hydrodynamic Forces

- (a) The lift and drag loads are obtained as

$$L = C_L \frac{1}{2} \rho \left\| \vec{V}_{rel}^4 \right\|^2 c \, dr, \quad (22)$$

$$D = C_D \frac{1}{2} \rho \left\| \vec{V}_{rel}^4 \right\|^2 c \, dr, \quad (23)$$

where ρ is the fluid density, and dr is the influence length (defined as the distance between the middle point between the current blade station and the next blade station and the middle point between the current blade station and the next blade station [3]).

- (b) The lift and drag forces are resolved in the normal (thrust) and tangential directions (to the relative velocity)

$$F_z = L \cos \phi + D \sin \phi, \quad (24)$$

$$F_y = L \sin \phi - D \cos \phi. \quad (25)$$

The torque is found as

$$T_y = F_y * \omega. \quad (26)$$

Warning

Currently, these forces are computed from quantities in system 4 and then output to be applied in system 3. There is some uncertainty, due to limited documentation in the literature, regarding the exact method by which this is performed. Further investigation is required.

Step 7: Quasi-Static Induced Velocity The next step is to update the quasi-static induced velocity

$$W_{qs,y,i}^4 = \frac{-N_B(F_L \cos \phi + F_D \sin \phi)}{4\pi L \rho F_P \left\| \vec{V}_0^4 + f_g \vec{n}(\vec{n} \cdot \vec{W}_{i-1}^4) \right\|}, \quad (27)$$

$$W_{qs,z,i}^4 = \frac{-N_B(F_L \sin \phi - F_D \cos \phi)}{4\pi L \rho F_P \left\| \vec{V}_0^4 + f_g \vec{n}(\vec{n} \cdot \vec{W}_{i-1}^4) \right\|}, \quad (28)$$

where N_B is the number of blades, and $\vec{n}$ is a unit vector in the direction of thrust (0;0;1). Note that both the tip 15, hub 16, and thrust 17 corrections are included here.

$$W_{qs,z,i}^4 = \{0; W_{qs,y,i}; W_{qs,z,i}\} \quad (29)$$

Warning

The coordinate frames here are somewhat confusing. Some sources evaluate W_{qs} in system 4, which seems reasonable since the blade frame is the natural reference [1]. However, projecting it directly onto the rotor frame can appear inconsistent. Branlard instead recommends first converting back to system 3 and then projecting along the rotor frame [9]. The confusion arises because W_{qs} is expressed in system 3 but then added to W , which is defined in system 4.

Step 8: Dynamic Wake

Including a dynamic inflow/wake model allows for a time delay in the quasi-static induced velocities (27, 28) before they reach equilibrium. The dynamic wake model proposed by Øye consists of two first-order filters and is used here. The time constants are found as:

$$\tau_1 = \frac{1.1}{1 - 1.3 \min(a, 0.5)} \cdot \frac{R_r}{\|V_0^4\|}, \quad (30)$$

$$\tau_2 = \left(0.39 - 0.26 \left(\frac{R_b}{R_r} \right)^2 \right) \cdot \tau_1. \quad (31)$$

The first-order differential equations, solved analytically, are given as follows:

$$H = W_{qs,i} + k \cdot \tau_1 * \frac{W_{qs,i} - W_{qs,i-1}}{\Delta t}. \quad (32)$$

$$W_{int,i} = H + (W_{int,i} - H) \exp\left(-\frac{\Delta t}{\tau_1}\right). \quad (33)$$

$$W0_i = W_{int,i} + (W_{i-1} - W_{int,i}) \exp\left(-\frac{\Delta t}{\tau_2}\right). \quad (34)$$

Note that H and W_{int} are intermediate values that have no physical meaning. k is an empirical constant and is typically assigned as 0.6.

Step 9: Skewed Wake

If the wind/current velocity is not in the rotor normal direction, which can be due to the rotor being yawed or tilted, a skewed wake correction should be applied. The skew angle is the angle between the wind/current and the rotational axis of the rotor

$$\chi = \arccos\left(\frac{\vec{n} \cdot V'}{\cos(\psi - \psi_0)}\right). \quad (35)$$

ψ_0 is the angle where the blade is deepest into the wake.

$$\psi_0 = \arctan\left(\frac{V0_y^2}{V0_z^2}\right) \quad (36)$$

The correction factor (Γ) can take several forms; however, this represents the simplest and most commonly used form.

$$\Gamma = 1 + \frac{R_b}{R_r} \tan\left(\frac{\chi}{2}\right) \cos(\psi - \psi_0) \quad (37)$$

The new velocity is obtained as a correction to the original velocity provided by the dynamic wake model.

$$W = \Gamma \cdot W0 \quad (38)$$

Caution

See [1] regarding V' and a more detailed description of this model. There are several nuances as to which blade and where on the blade the mean quantities are computed.

3.3 Ocean Engineering Toolbox Theory

The derivation of the relevant wave hydrodynamic, environmental, mooring, and multibody dynamic formulations is provided in [13], while the specific implementation within the OET is detailed in [11]. No theoretical framework is provided for the gearbox and generator models.

[OceanEngineeringToolbox](#) / [src](#) / [Processing](#) / [Tidal](#) / 

Figure 2: Directory to the Tidal Code in the Ocean Engineering Toolbox GitHub Repository

4 Software

The code is available in the GitHub repository `OceanEngineeringToolbox` (tidal branch), which is a branch of the primary Ocean Engineering Toolbox codebase. The specific tidal implementation is located in the directory illustrated in Fig. 2.

This section addresses the software implementation of the BEMT algorithm and the associated Modelica models, highlighting the challenges of developing the algorithm within Modelica. It is organized into Subsection 4.1, which presents the MATLAB implementation of the BEMT algorithm, and Subsection 4.2, which outlines the preliminary multibody, generation, and FMU models in Modelica. However, before presenting these subsections, the key challenges encountered are introduced.

1. BEMT Algorithm Incompatibility with the Modelica Language

Quasi-steady/steady BEM iterates/relaxes the induction factors to determine the aero/hydrodynamic loads at the current step, whereas unsteady BEM does this through a time evolution (i.e. dependence on state information from the previous time step). Modelica is primarily designed to represent continuous systems, which makes it less compatible with this approach. While discrete variables can be represented, doing so requires defining specific sampling rates — a process that is feasible and necessary in this case. However, this significantly constrains the choice of numerical solvers to fixed-step methods and triggers an event at each time step, thereby greatly reducing computational efficiency. Moving forward, there appear to be three alternatives:

(a) Modelica Workaround

- **Pros:**
 - Everything remains within Modelica.
- **Cons:**
 - Discrete variables can be represented and sampled at specified intervals (functionality developed for embedded systems applications).
 - Each discrete event triggers an “event” in the integrator, causing a restart and significantly reducing computational efficiency.

(b) Co-simulation with MATLAB (FMU)

- **Description:** Develop the unsteady BEMT algorithm in MATLAB/Simulink, export it as an FMU, and run a co-simulation with the OET (preferably in a Modelica environment).
- **Pros:**
 - Portable: once exported as an FMU, the code can be easily interfaced with Modelica.
 - Efficient: exported FMUs are compiled as C/C++, avoiding computational inefficiencies of interpreted MATLAB code.
 - Flexible: allows rapid development and adaptation to custom needs.
- **Cons:**
 - Limited by communication step size: if fixed, numerical solvers must also be fixed-step.
 - Simpler compared to AeroDyn, as it is a custom implementation.

(c) Coupling with AeroDyn

- **Description:** Two possibilities: wrap AeroDyn as an FMU (challenging) or run it via a dynamic link library (DLL).
- **Pros:**

- Leverages advanced modelling capabilities and familiarity with AeroDyn, enhancing credibility.
- Integrates with OET in Modelica for multibody dynamics and multi-physics (e.g., generator modelling).
- Eliminates the need to develop and validate a custom BEMT algorithm.
- **Cons:**
 - Significant learning curve: requires confidence in AeroDyn, experience with C, and setting up a DLL.
 - Not fully portable: likely restricted to Windows; Linux compatibility may be limited.
 - Interfacing with AeroDyn can be challenging.

Option (b), **Co-simulation with MATLAB (FMU)**, was identified as the most promising approach with the fewest limitations, and it is therefore the method adopted in this work.

2. Limited FMU Importing Capabilities within OpenModelica

Originally, the plan was to develop the BEMT algorithm in MATLAB/Simulink (due to Limitation 1), export it as an FMU, and run a co-simulation in OpenModelica. However, given the FMU import limitations of OpenModelica (described below), this approach is not feasible. Instead, the BEMT algorithm is implemented in Simulink, and the Modelica model is exported as an FMU to run a co-simulation within the Simulink environment, leveraging Simulink’s FMU import block.

OpenModelica

- **Import:**
 - Supports ME FMUs 1.0 and 2.0.
 - CS FMUs 1.0 with 2.0 is under development.
 - FMI 3.0 support is planned for late 2025 or early 2026, but not guaranteed (noted for a response to an issue on the Modelica Standard Library GitHub repository).
- **Export:**
 - Can export FMUs with interfaces for both ME and CS FMI 2.0.
- **Limitations:**
 - ME FMUs from Simulink cannot be easily simulated because OpenModelica’s solver is rigid regarding the formulation of system of equations; models must adhere to strict regulations that any non-Modelica FMU likely cannot conform to.
 - Therefore, a BEMT algorithm implemented as an ME FMU cannot run in OpenModelica.

Simulink

- **Import:**
 - ME FMUs: FMI 1.0, 2.0, and 3.0.
 - CS FMUs: FMI 1.0, 2.0, and 3.0.
- **Export:**
 - ME FMUs: FMI 2.0 and 3.0.
 - CS FMUs: FMI 2.0 and 3.0.
 - Does not support FMI 1.0.
- **Opportunities:**
 - Modelica multibody, hydrodynamic, and generator models can be exported as an FMU and executed with a CS configuration in Simulink.

4.1 MATLAB BEMT Algorithm

1. Algorithm Description

Before running the algorithm, the airfoil polar data must be extracted. This is accomplished using the function `parseXFoilPolar.m`, which processes the relevant information and saves it in a `.mat` file. The function takes as input the `.txt` polar data file generated by XFoil. A template for calling this function is provided in `XFoil_process.m`. The function is compatible with both four-digit and five-digit airfoils.

Once the airfoil data has been preprocessed, the algorithm can be executed. While specific implementation details will be discussed later, the high-level structure of the BEMT algorithm is outlined in Section 3.2, with the corresponding code implementation shown in Algorithm 1.

Algorithm 1 Unsteady BEMT Algorithm

```

1: Step 0: Initialize blade/turbine parameters, blade positions, and velocities
2: for each time step do
3:   for each blade do
4:     for each blade element do
5:       Step 1: Update incoming wind profile using (8) (optional tower shadow)
6:       Step 2: Compute relative velocity using (9)
7:       Step 3: Compute flow angle, angle of attack, and Reynolds number using (11), (12), and (13)
8:       Step 4: Apply tip and hub loss corrections (15), (16), and thrust correction (17)
9:       Step 5: Compute lift and drag coefficients (with dynamic stall correction) using (18) with
inputs from (19), (20), and (21)
10:      Step 6: Compute aerodynamic forces and torque using (22)–(26)
11:      Step 7: Update quasi-static induced velocity using (27)–(29)
12:      Step 8: Apply dynamic wake model using (30)–(34)
13:      Step 9: Apply skewed wake correction using (35)–(38)
14:    end for
15:  end for
16: end for

```

2. Code Structure

The algorithm was developed using OOP principles, with the codebase organized into distinct classes:

- *simulationSettings* (*sim*) — defines the simulation settings
- *fluidProperties* (*fluid*) — defines fluid properties
- *incomingWind* (*wind*) — specifies the characteristics of the incoming wind
- *coordTransformations* (*rot*) — handles kinematic rotations
- *aeroCorrections* (*cor*) — applies aerodynamic and hydrodynamic corrections
- *windTurbine* (*turb*) — defines the wind turbine characteristics
- *variablesBEMT* (*var*) (MATLAB version only) — stores variable history classes (objects)

These classes are instantiated within the BEMT algorithm, which is implemented either as a Simulink block (Simulink version) or called directly by an input file (MATLAB version). A combination of handle (pass by reference), value (pass by value), and System (Simulink MATLAB code block) classes are utilized.

3. MATLAB and Simulink Branches

The original code was developed entirely in MATLAB. However, since both exporting the BEMT algorithm as an FMU and importing the Modelica model as an FMU for co-simulation require the Simulink environment, the MATLAB code had to be adapted into compatible Simulink blocks. This resulted in a second branch of the code.

Both branches remain in use. The Simulink version is actively maintained with updates and comments and serves as the primary tool for simulation. The MATLAB version, while no longer as actively maintained, remains valuable for debugging because it allows the algorithm to be stepped through more easily and offers greater flexibility in visualizing results. As will be discussed in Section 5.2, debugging remains essential due to existing errors in the algorithm. Subsection 4.1.1 describes the MATLAB debugging approach, while Subsection 4.1.2 outlines the Simulink simulation implementation.

4.1.1 Debugging BEMT Algorithm

The directory containing the code discussed in this section is shown in Fig. 3.

OceanEngineeringToolbox / src / Processing / Tidal / BEMAlgorithms /

Figure 3: Directory to the Debugging Code in the Ocean Engineering Toolbox GitHub Repository

The original code was designed to support debugging of the BEMT algorithm, with most intermediate states (for each time step, blade, and blade element) preserved to facilitate troubleshooting. It is also used to enable batch simulations for comparison with validation results, although this functionality should eventually be ported to the Simulink model. Overall, the MATLAB implementation may be regarded as a rough draft, while the Simulink model should be considered the most up-to-date version.

Figure 4 illustrates the MATLAB environment for the debugging algorithm. In particular, one class (`unsteadyBEMTAlgorithm.m`) and three scripts (`unsteadyBEMTm.m`, `debugging.m`, and `batchBEMT.m`) are relevant. The remaining classes were introduced in Section 4.1.

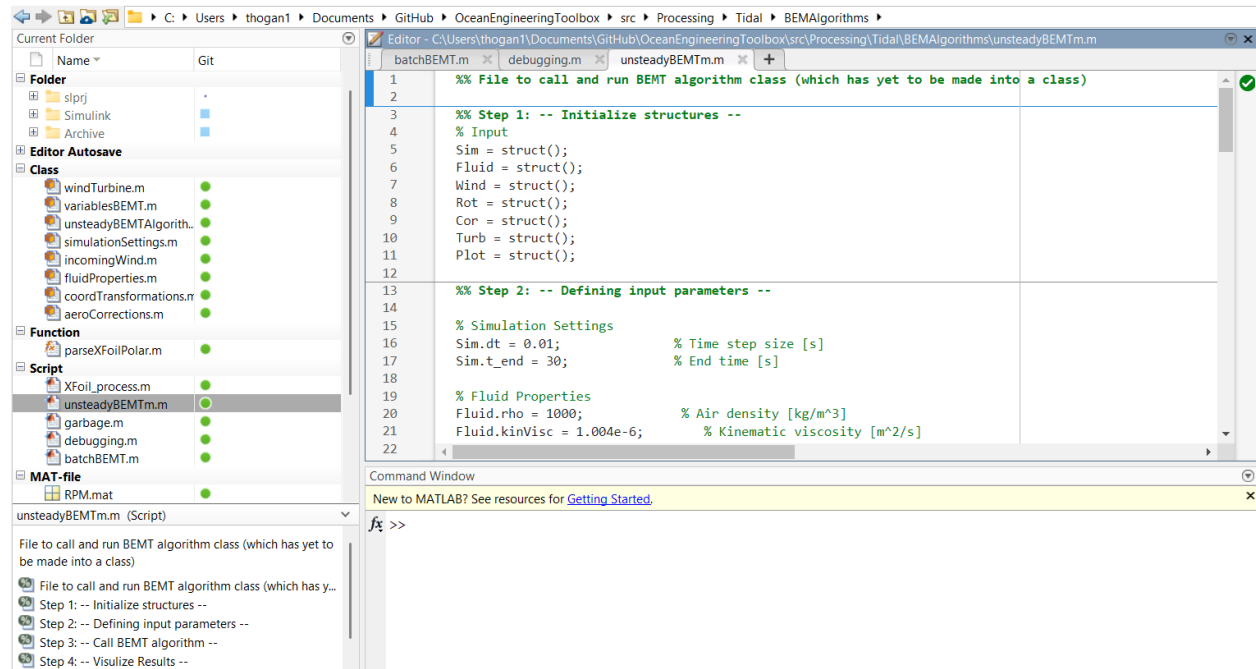


Figure 4: MATLAB Environment - Debugging Algorithm

The `unsteadyBEMTAlgorithm.m` class is responsible for calling all the listed classes and implementing the BEMT algorithm, forming the core of the codebase. The `unsteadyBEMTm.m` script defines the simulation parameters and executes the simulation. In this case, airfoil data preprocessing is performed within the algorithm, so the airfoil `.txt` file must be provided. The `debugging.m` script extracts data stored in the `variablesBEMT` (`var`) class and plots it for debugging a single element on a single blade at a time, which is extremely useful for troubleshooting. The `batchBEMT.m` script supports validation by calling `unsteadyBEMTm.m` for each test case being compared, which in this study includes twelve cases.

4.1.2 Simulink BEMT Algorithm

The directory containing the code discussed in this section is shown in Fig. 5.

[OceanEngineeringToolbox / src / Processing / Tidal / BEMAlgorithms / Simulink /](#)

Figure 5: Directory to the Simulink Code in the Ocean Engineering Toolbox GitHub Repository

The Simulink model should be considered the most up-to-date and maintained version; it is the one to be referenced when moving forward.

Figure 6 illustrates the MATLAB environment for the Simulink algorithm. In particular, two classes (`unsteadyBEMTSystemAlgorithm.m` and `unsteadyBEMTSystem.m`), one script (`XFoil_process.m`), and two Simulink models (`unsteadyBEMTs.slx` and `FMUunsteadyBEMTs.slx`) are relevant. The remaining classes were introduced in Section 4.1.

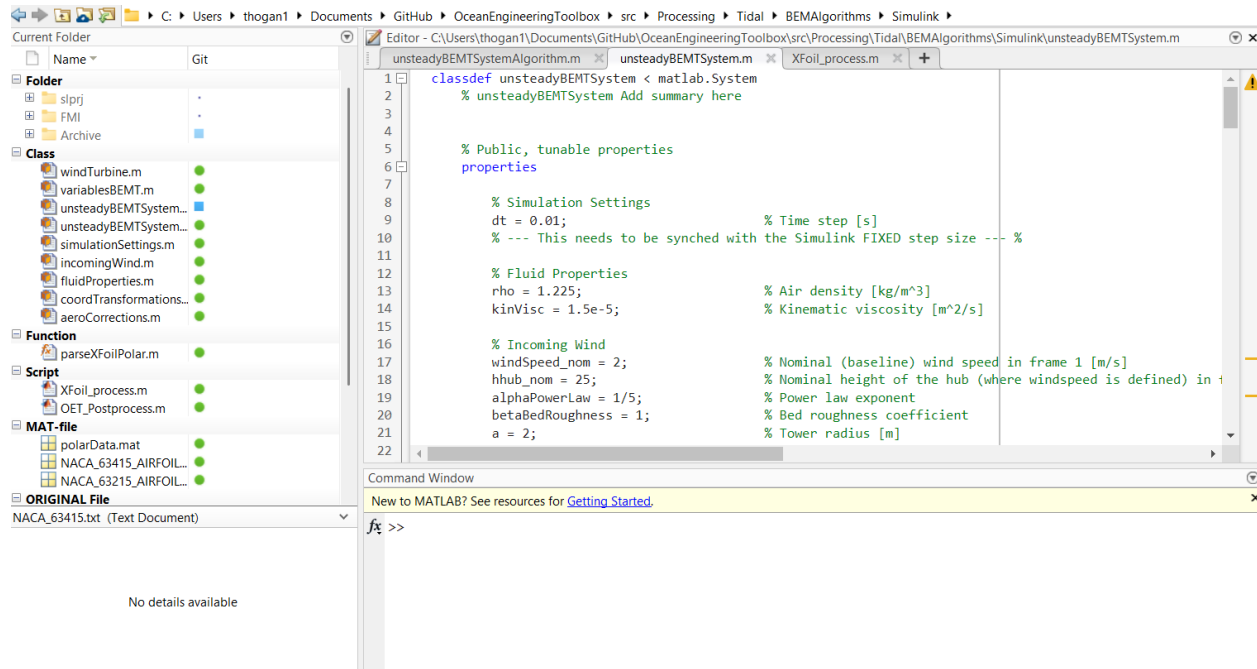


Figure 6: MATLAB Environment - Simulink Algorithm

The `unsteadyBEMTSystemAlgorithm.m` class is responsible for calling all relevant classes and implementing the BEMT algorithm, forming the core of the codebase. The main difference in this version is that the manual time-stepping loop has been removed and is now handled by Simulink. The `unsteadyBEMTSystem.m` class contains the code for the Simulink *MATLAB System* block, which defines the simulation parameters and executes the simulation. In this case, airfoil data preprocessing is performed **externally**, so the airfoil `.mat` file must be provided. The `XFoil_process.m` script should be executed to extract the airfoil polar data and save it in the `.mat` file. A useful description of the setup for the Simulink *MATLAB System* block can be found here. The Simulink model `unsteadyBEMTs.slx` implements the algorithm in Simulink, where `unsteadyBEMTSystem.m` is called directly, as shown in Fig. 7. Similarly, `FMUunsteadyBEMTs.slx` is an FMU of the algorithm, as shown in Fig. 8. This is achieved by importing `unsteadyBEMTSystem.m` into a Simulink environment and exporting the model as an FMU. However, this requires the following MATLAB/Simulink Add-Ons to be installed:

1. MATLAB Coder

2. MATLAB Compiler
3. Simulink Coder
4. Simulink Compiler
5. FMU Builder for Simulink
6. MATLAB Support for MinGW-w64 C/C++/Fortran Compiler

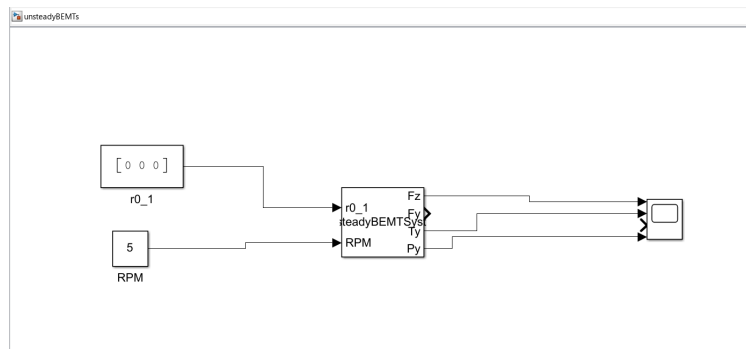


Figure 7: MATLAB Environment - Simulink Algorithm System Block

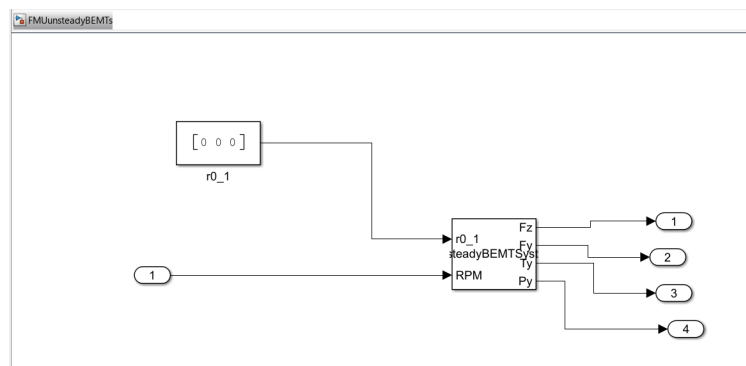


Figure 8: MATLAB Environment - Simulink Algorithm FMU

4.1.3 Validation Scripts

The directory containing the code discussed in this section is shown in Fig. 9.

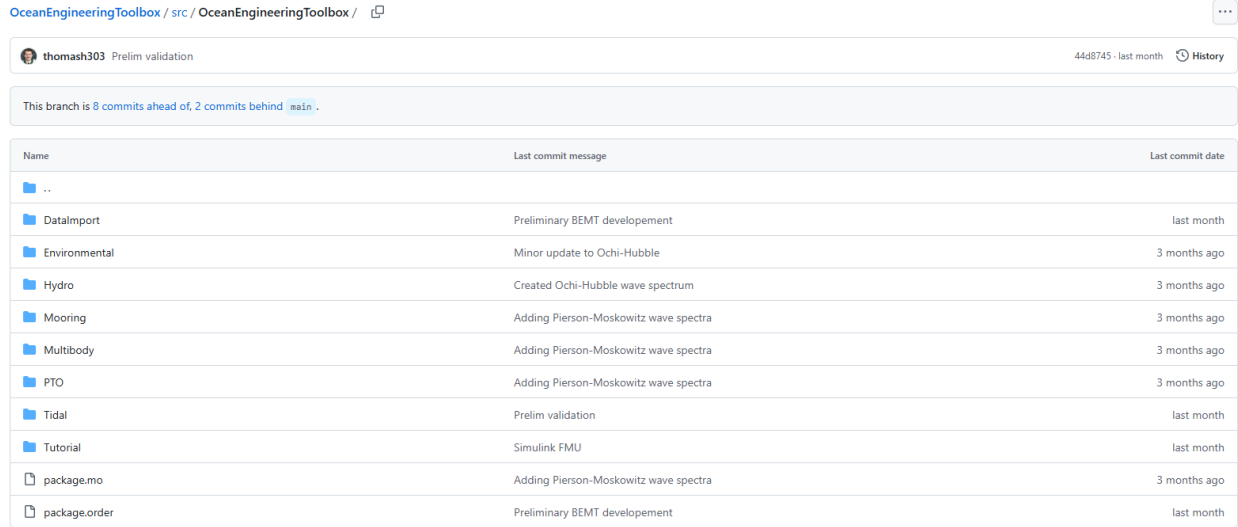
[OceanEngineeringToolbox / applications / Validation / Tidal /](#) 

Figure 9: Directory to the Validation Code in the Ocean Engineering Toolbox GitHub Repository

In particular, one script, `Plotting_TTT_Data.m`, is relevant; it extracts the necessary data and generates the corresponding plots.

4.2 Modelica Models

The directory containing the code discussed in this section is shown in Fig. 10. To open the model in Modelica, navigate to this level of the directory and open the `package.mo` file.



Name	Last commit message	Last commit date
..		
DataImport	Preliminary BEMT development	last month
Environmental	Minor update to Ochi-Hubble	3 months ago
Hydro	Created Ochi-Hubble wave spectrum	3 months ago
Mooring	Adding Pierson-Moskowitz wave spectra	3 months ago
Multibody	Adding Pierson-Moskowitz wave spectra	3 months ago
PTO	Adding Pierson-Moskowitz wave spectra	3 months ago
Tidal	Prelim validation	last month
Tutorial	Simulink FMU	last month
package.mo	Adding Pierson-Moskowitz wave spectra	3 months ago
package.order	Preliminary BEMT development	last month

Figure 10: Directory to the Modelica Code in the Ocean Engineering Toolbox GitHub Repository

Figure 11 illustrates the Modelica directory to the tidal portion of the code. In particular, two models (`sampleFMU.mo` and `turbine.mo`) are relevant

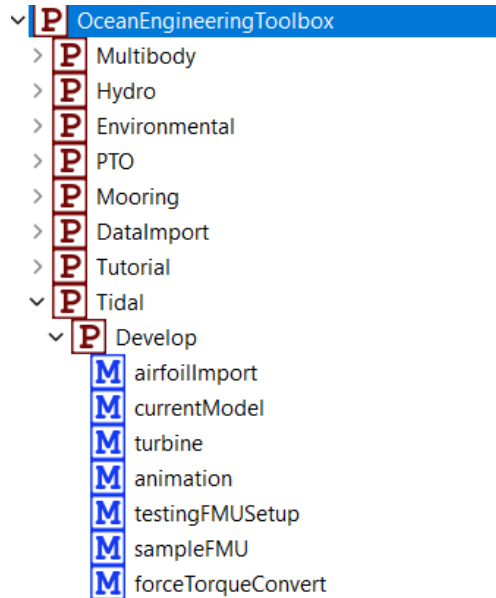


Figure 11: Directory to the Tidal Portion in the Ocean Engineering Toolbox Codebase

The `sampleFMU.mo` model (shown in Fig. 12) provides a simple representation with the appropriate inputs and outputs for co-simulation with the BEMT algorithm in Simulink. This model can be directly exported as an FMU. The `turbine.mo` model (shown in Fig. 13) is intended primarily for demonstration purposes, illustrating how a more complete turbine could be represented in Modelica, though it does not perform an actual simulation.

The number of bodies is based on the wind turbine coordinate systems defined in Fig. 1, but can be adapted for a convenient tidal turbine configuration. The platform is represented as a *HydrodynamicBody* from the OET, which enables the application of hydrodynamic forces to the floating structure, and is connected to a *linearMooring* model from the OET. The model is formulated in 6-DoF, with most components taken from the Multibody library in the MSL.

If generator models are added to the rotor, their mechanical interfaces use 1D rotational flanges, which are incompatible with standard multibody connections. To resolve this, the nacelle (non-rotating) and hub (rotating) are connected by a revolute joint, enabling relative rotation. By enabling the `useAxisFlange` flag on this joint, generator models can be connected through their 1D rotational flanges. In this example, a gearbox and a permanent magnet synchronous generator are implemented, though any generator model from the MSL (or a custom model with the same interface) could be used.

It should be noted that when electrical dynamics are included, the timescales of the electrical and multibody systems differ significantly, which may affect the simulation. Finally, turbine motion can be visualized using *FixedShape* models with associated `.stl` geometry files. To ensure consistency, `.stl` files should be exported with the coordinate frame defined at the component's centre of mass.

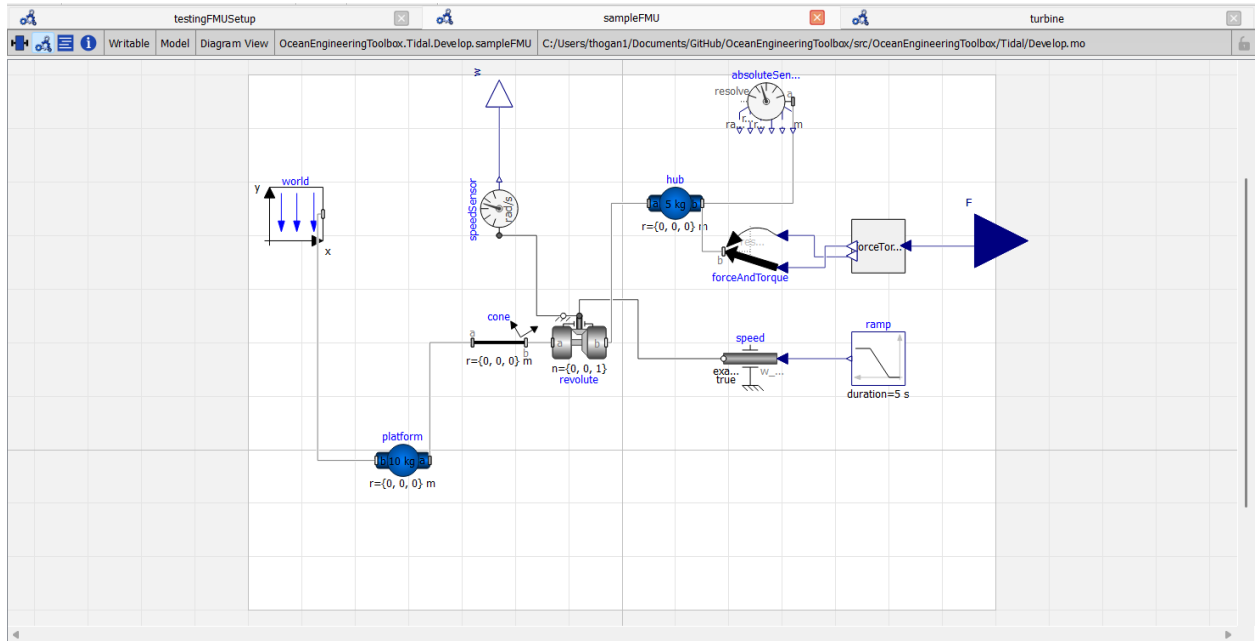


Figure 12: Sample Modelica FMU Configuration

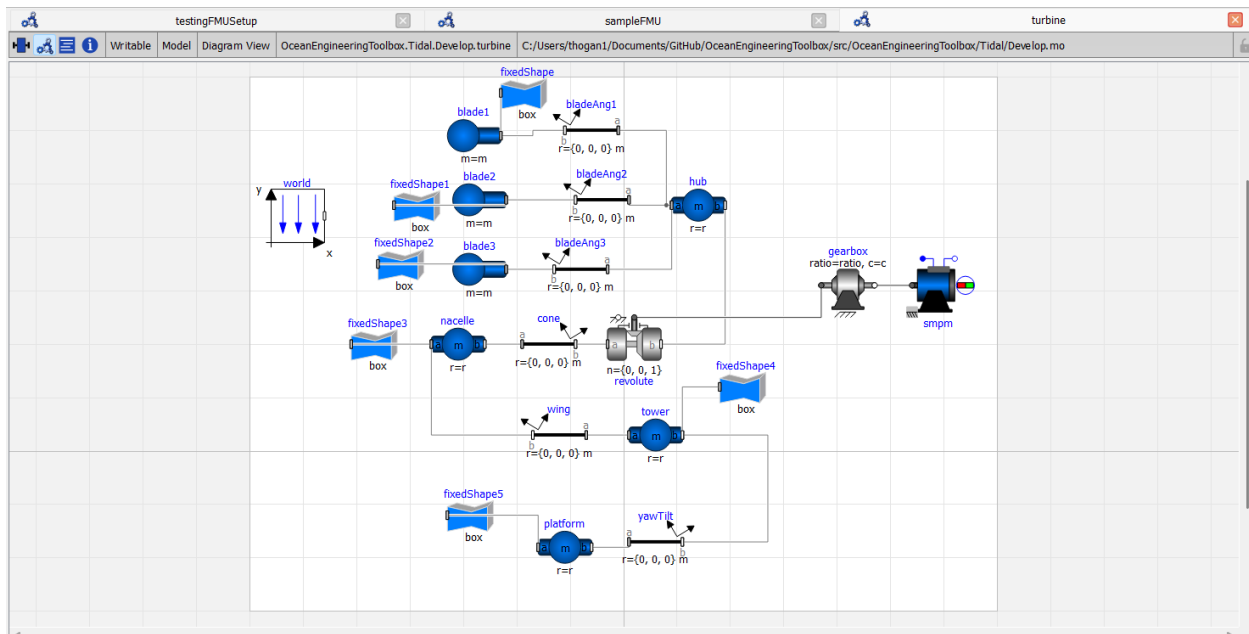


Figure 13: Sample Modelica FMU Configuration

5 Debugging and Validation

The following section presents selected debugging results to provide insight into the code's operation and potential clues to the underlying issues. While most components appear to function as intended, a few problems propagate errors. Additionally, the BEMT algorithm is compared with experimental results from a medium-scale towing tank.

5.1 Debugging

Once the `unsteadyBEMTm.m` script has been executed and the simulation data saved to variables, the `debugging.m` file can be run to visualize the results. Several plots that proved particularly valuable are presented, accompanied by a brief commentary on their validity. The turbine and airfoil characteristics used in this study correspond to those employed in the experimental testing.

The blade azimuthal and element positions are shown in Fig. 14. The results appear reasonable, as the sinusoidal motion is consistent with a rotating body. Furthermore, the same element at different positions is out of phase, corresponding to their angular position offsets.

Element 1 — Azimuth & Blade Positions

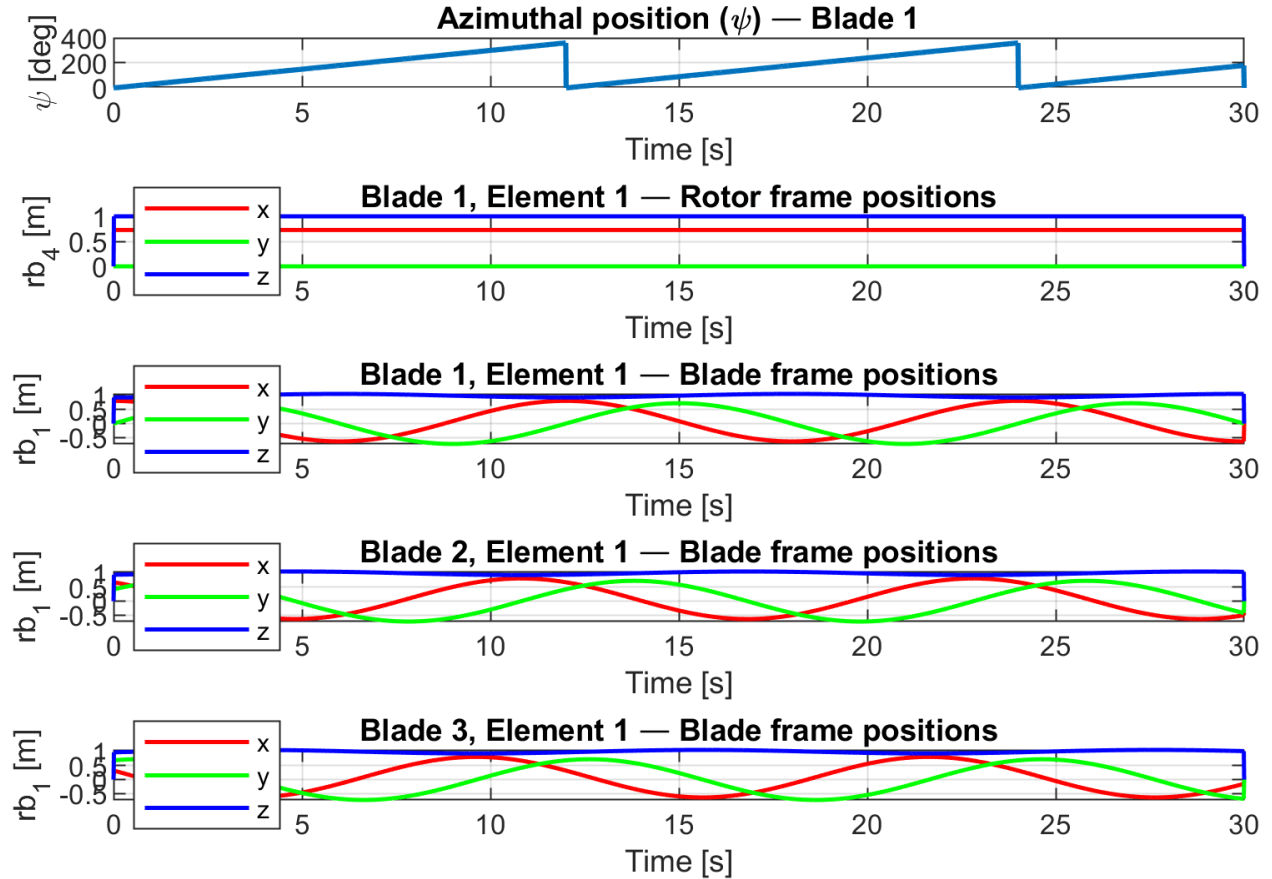


Figure 14: Blade Positions - Debugging Visualization

The incoming wind is a function of the instantaneous blade element height. Since the blade element follows a circular trajectory, it follows that the corresponding incoming wind would see a sinusoidal response, as shown in Fig. 15.

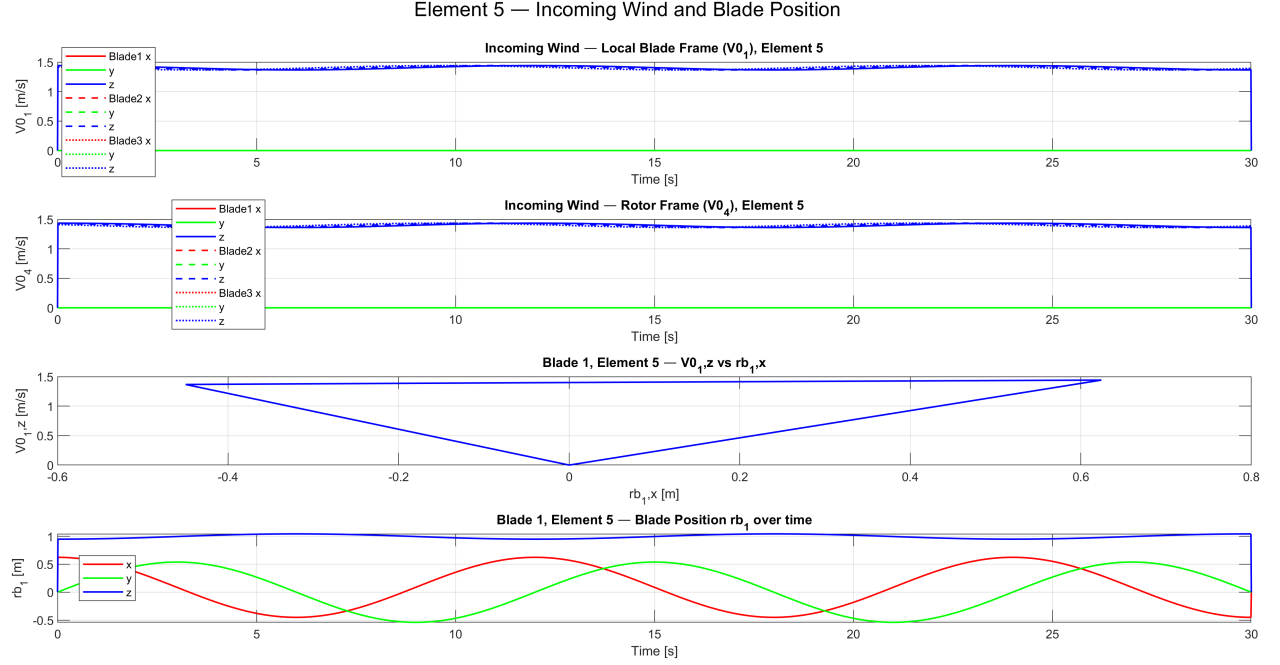


Figure 15: Incoming Wind - Debugging Visualization

The constituent components of the relative velocities appear reasonable in Fig. 16. Here, element 1 refers to the tip. It is observed that as the rotational velocity increases, this component grows disproportionately compared to the others and becomes dominant, resulting in unusual behaviour in the induced velocities.

As reported previously, the results in Fig. 17 also appear reasonable. However, as the angular velocity increases, some unusual behaviour becomes apparent, with the angle of attack reaching unreasonably large values.

Again, the induced velocity results appear reasonable, as shown in Fig. 18, which corresponds to a point near the root. However, at high rotational speeds, unusual behaviour emerges, particularly for elements near the tip.

The results shown in Fig. 19 illustrate the torque at the tip element of a single blade, which exhibits the highest torque on that blade. For the total torque, however, contributions from all elements on all blades must be summed.

Blade 1, Element 2 — Velocities

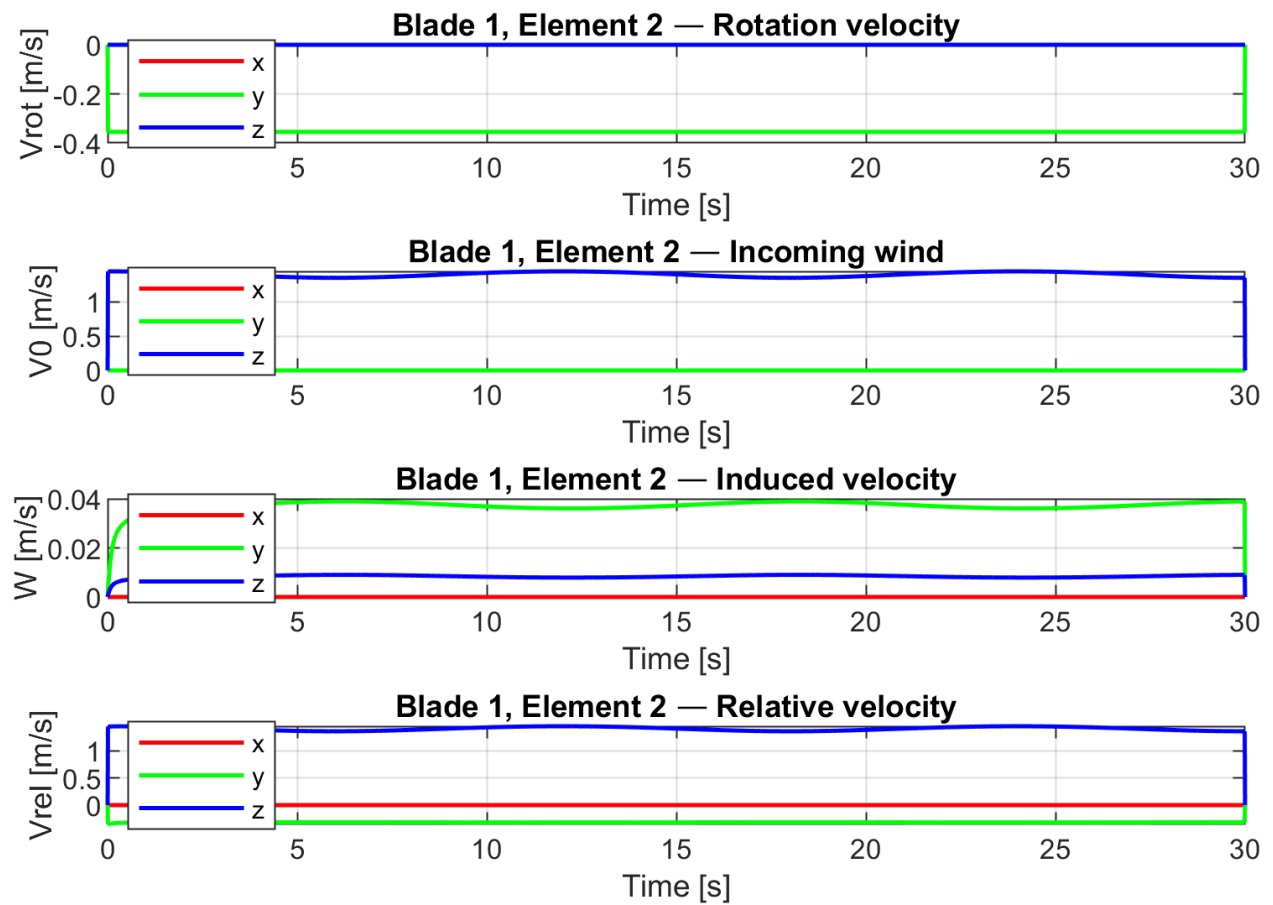


Figure 16: Relative Velocity - Debugging Visualization

Blade 1, Element 2 — W_{qs} , V_{rel}_4 & Alpha

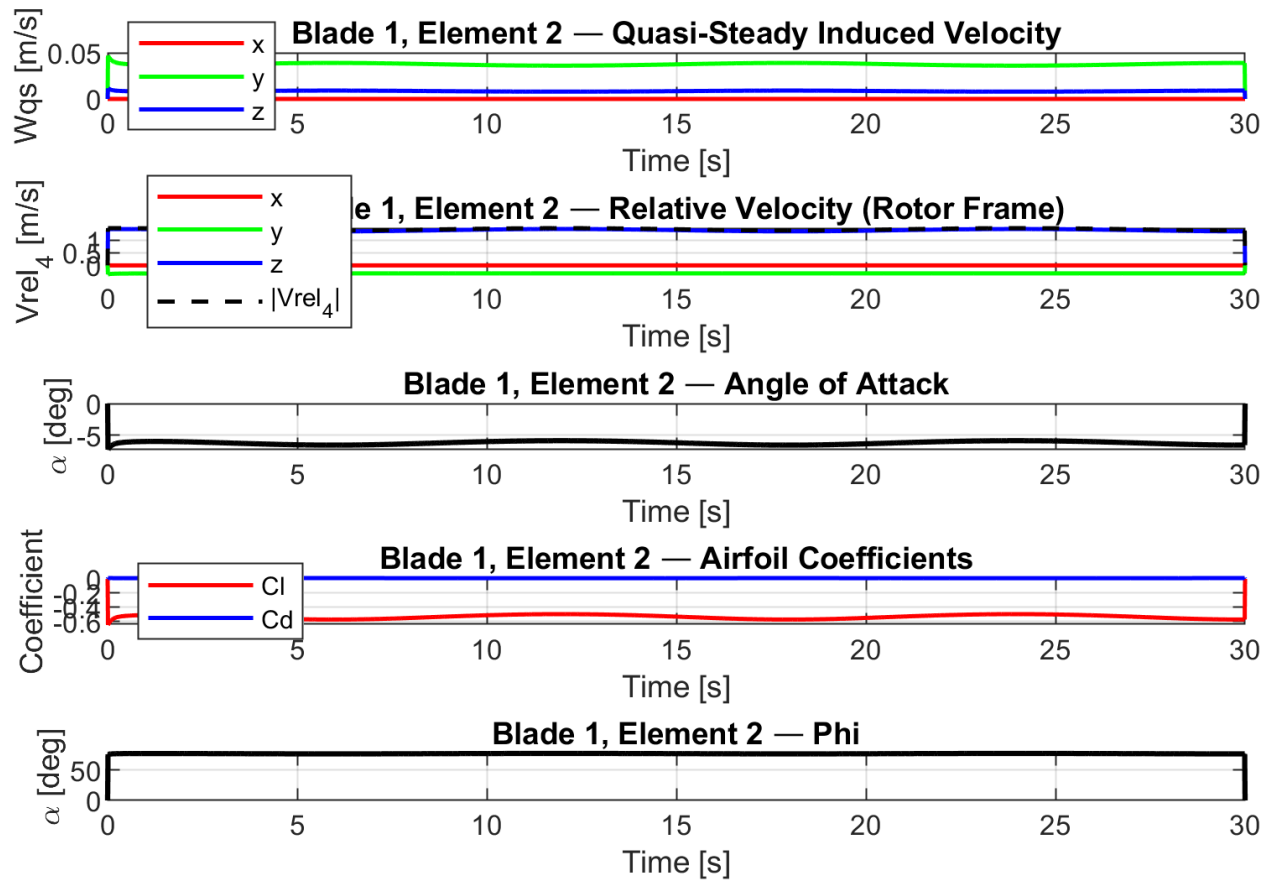


Figure 17: Angle of Attack - Debugging Visualization

Blade 3, Element 15 — W_{qs} , W & Γ

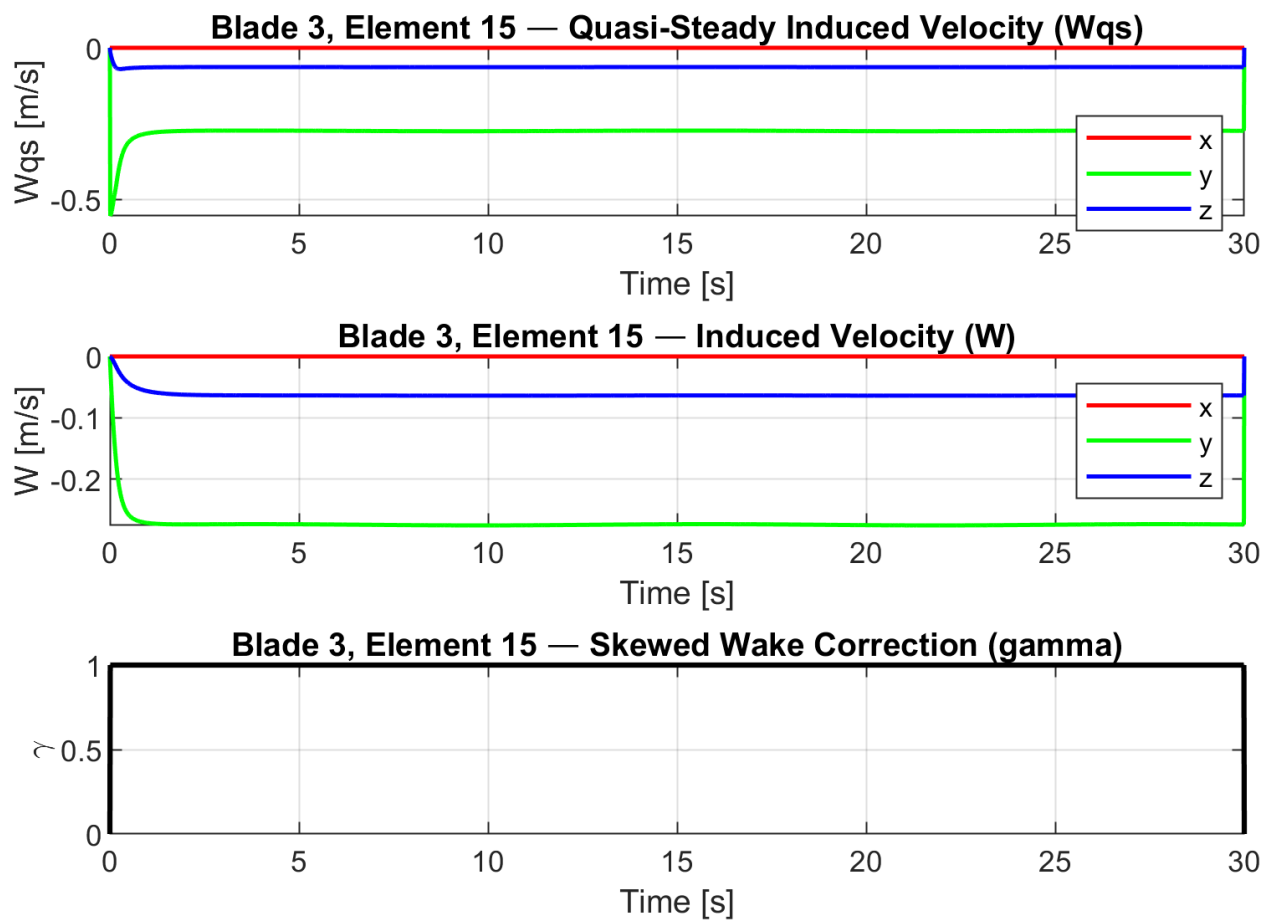


Figure 18: Induced Velocity - Debugging Visualization

Blade 1, Element 1 — p_y , p_z , T_y

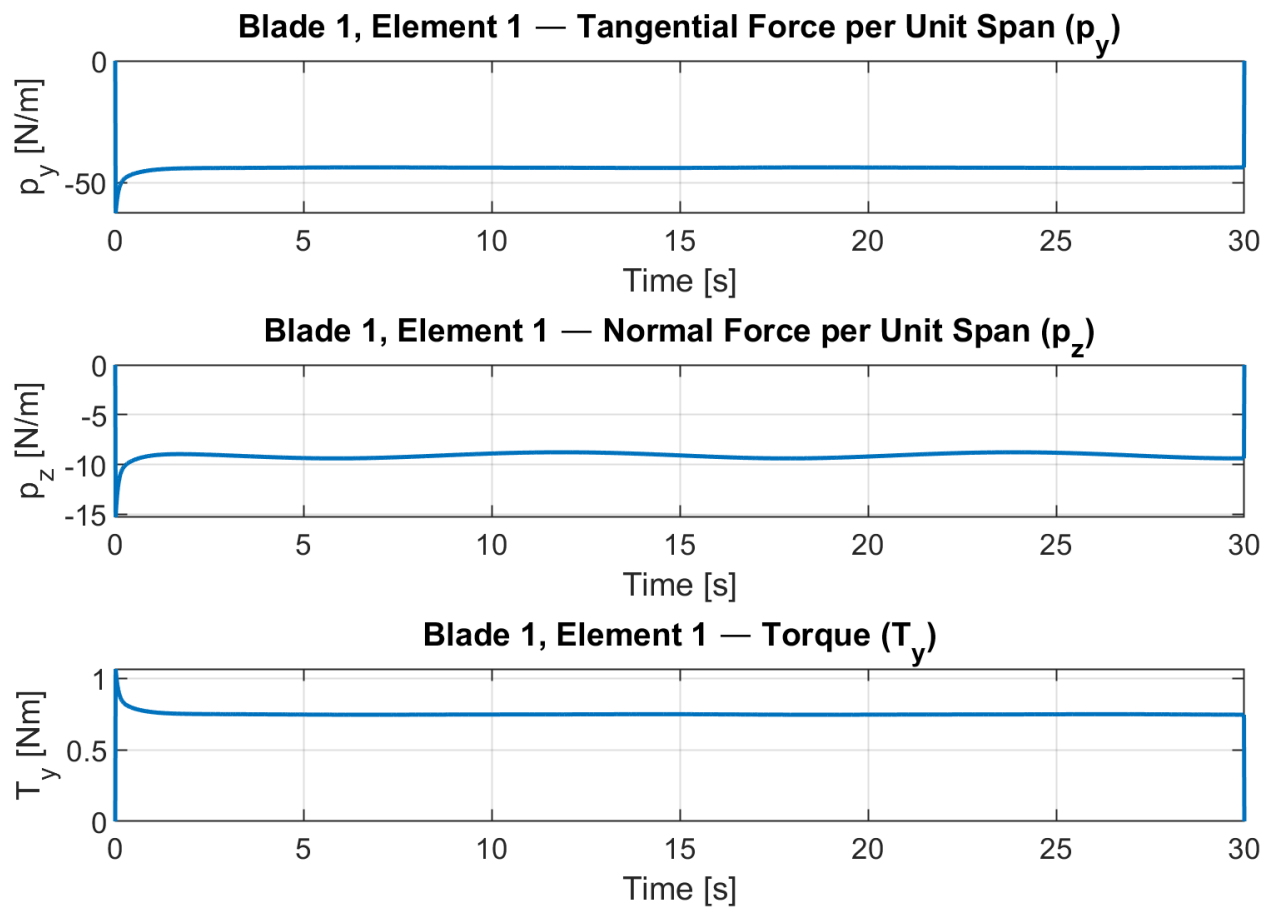


Figure 19: Aero/Hydrodynamic Loads - Debugging Visualization

5.2 Validation

Experimental validation was conducted using medium-scale towing tank data. Additional details about this experiment, including CAD files, blade profile data, and a presentation summarizing the work, are provided in the directory listed in Section 4.1.3.

The BEMT algorithm was compared to the experimental data for a single carriage speed across 11 different rotor speeds, evaluating: a) angular velocity, b) thrust force, c) torque, and d) mechanical power in Fig. 20. In all tests and across all categories, the BEMT algorithm consistently overestimated the experimental results. There is generally an acceptable margin of error; however, these results exceed the reasonable limits. The cause(s) of this discrepancy remain uncertain, and further investigation is required to troubleshoot the algorithm. This model can be tested using only the BEMT algorithm, without requiring any Modelica model.

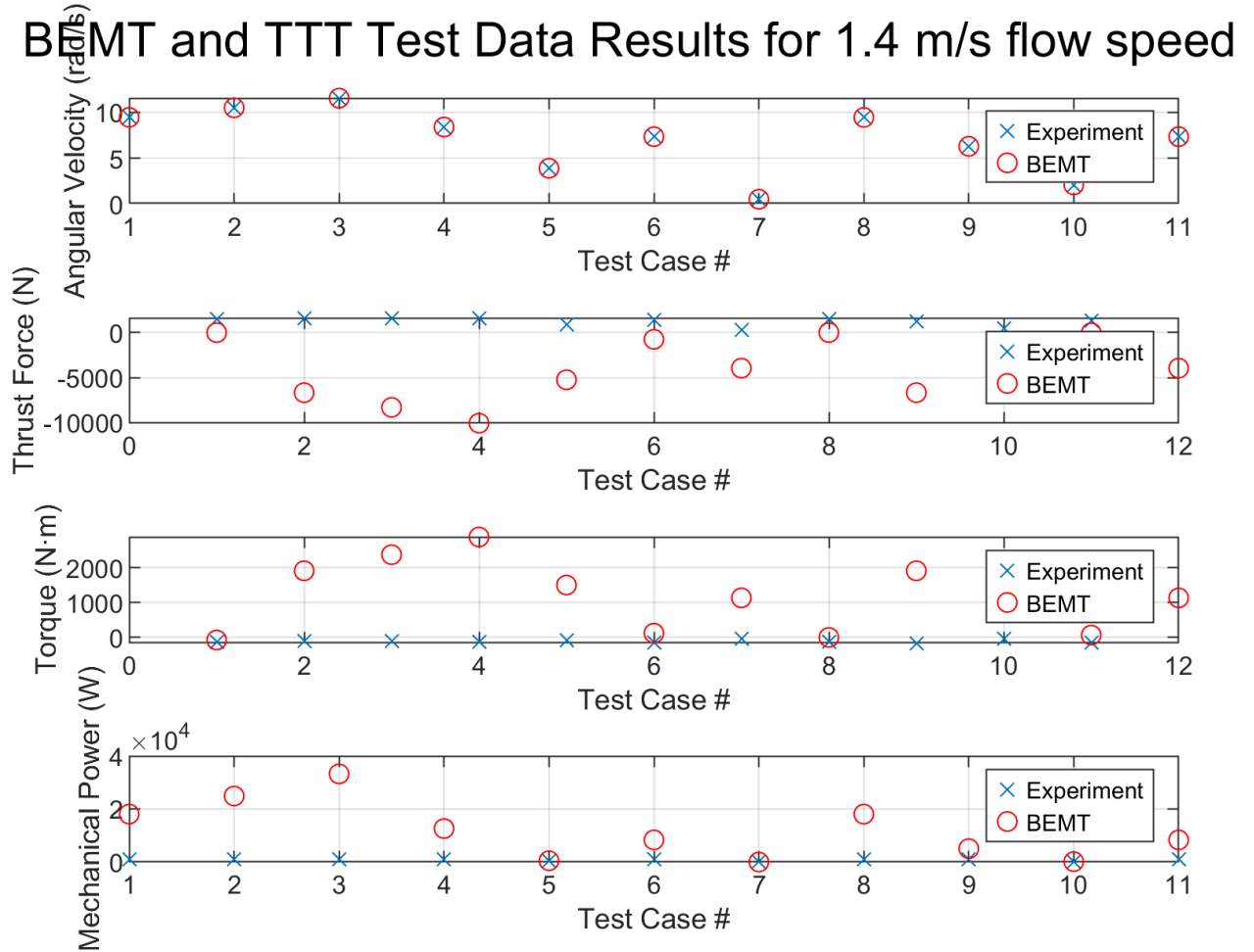


Figure 20: BEMT Validation with Medium-Scale Towing Tank Experimental Data

6 Conclusions and Future Work

The work summarized in this report examined the feasibility of modelling a tidal stream generator in Modelica. Although several limitations were encountered, including (a) the inability to develop the BEMT algorithm in Modelica and (b) the limited capabilities of OpenModelica to import FMUs, the study was able to identify a potential pathway forward. Specifically, challenge (a) was addressed by developing the BEMT algorithm in MATLAB, while challenge (b) was addressed by executing the co-simulation in Simulink.

Although this approach relies on a less mature BEMT algorithm that still contains several bugs, the ability to leverage FMUs for co-simulation with Modelica code is invaluable. Modelica enables multi-physics simulations, which are essential for resource-to-wire modelling and, consequently, for control co-design. Therefore, despite the current immaturity of the codebase, it demonstrates substantial potential for the numerical modelling and simulation of tidal stream generators.

6.1 Future Work

This work was primarily exploratory, and significant tasks remain to be completed. These are outlined below:

- The current formulation neglects the contributions of the infinite-frequency added mass and the wave-induced water particle velocities. The work in [14] presents a formulation to address the former. This is not to suggest that neglecting these effects is unreasonable; these contributions may not need to be accounted for, but this has yet to be investigated.
- The current tower shadow model for the incoming wind is highly problematic and requires a substantial overhaul. Furthermore, since this model is designed for wind, it is unclear whether it would be applicable to tidal applications. Furthermore, the incoming wind profile must be adjusted to account for the variations in the platform position. In addition, the skewed wake model should be thoroughly reviewed, with careful consideration of its relevance to tidal scenarios.
- As highlighted in Section 5.2, there are several unidentified uncertainties and issues within the code that require a thorough review and debugging. Although this process has already been undertaken, some issues persist. Prior to continuing experimental validation, a code-to-code verification with [10] could provide additional insight into these remaining issues.
- Following successful code verification, experimental validation should be conducted again to confirm the accuracy and reliability of the model.
- This work alternates between radians and degrees for visualization and computation; ensuring consistency in future work would be valuable.
- Construct a complete Modelica multibody and hydrodynamic model for the tidal turbine, building upon the example shown in Fig. 13. This process requires obtaining the corresponding hydrodynamic data for the platform. Once the model is developed, it must be formatted to ensure FMU compatibility and include the proper input/output interfaces to interact with the BEMT algorithm.
- To fully leverage the benefits of Modelica, particularly its multi-physics modelling capabilities, the gearbox and electric generator models should be integrated into the Modelica turbine model. This approach could enable further experimental validation, using electrical power capture as the reference instead of mechanical power capture.
- There remains a coordinate system inconsistency between the MSL and the BEMT algorithm, specifically in the definition of the coordinate axes. It is also necessary to verify that the kinematic rotations yield the same positions when using the manual rotation matrices defined in the BEMT algorithm and those prescribed in the MSL.
- Ensure that the forces and induced velocities are evaluated and applied in the correct coordinate system, as there is currently some ambiguity in their implementation.
- The current work neglects blade elasticity, which could become significant in cases where the blades experience large deformations. This aspect should be further investigated.

6.2 Recommended Literature

During the literature review and throughout this work, several references proved particularly valuable. These will be listed below, along with a brief description of their relevance and contribution.

- [1] is the best resource for reviewing the fundamental theory behind both the steady and unsteady BEMT algorithms; it is pretty universally cited. [10] presents the exact same content (almost verbatim), but in a very condensed manner and includes a case study that implements an unsteady algorithm and presents many of the results. That makes it a strong candidate for code-to-code verification prior to experimental validation. [9] provides code corresponding to the algorithm in [1], with the complete steady implementation and portions of the unsteady code. [3] also provides a clear and concise description of the algorithm.
- [15] develops a dynamic model for a wind-wave-tidal platform and briefly outlines the governing equations needed for such a study.
- [14] presents a formulation to account for the added mass of the fluid for a tidal turbine application in FAST v8 (which I believe this was eventually merged into OpenFAST).
- [16] and [17] propose coupling frameworks for wind turbines by modelling hydrodynamics and multi-body dynamics in Modelica while linking to AeroDyn and MoorDyn via DLLs. While relevant, the quality of these works is relatively low, and their methodology remains unclear.

References

- [1] M. O. L. Hansen, *Aerodynamics of wind turbines*, 2nd ed. London: Earthscan, 2010.
- [2] AeroDyn Users Guide and Theory Manual. [Online]. Available: <https://openfast.readthedocs.io/en/main/source/user/aerodyn/index.html>
- [3] Ashes: Aerodynamic loads. [Online]. Available: <https://www.simis.io/docs/aerodynamics-aerodynamic-loads>
- [4] OrcaFlex: Turbines. [Online]. Available: <https://www.orcina.com/webhelp/OrcaFlex/Content/html/Turbines.htm>
- [5] MOST Manual. [Online]. Available: <https://wec-sim.github.io/WEC-Sim/main/most/index.html>
- [6] P. Fritzson, *Principles of object-oriented modeling and simulation with Modelica 3.3: a cyber-physical approach*. John Wiley & Sons, 2015.
- [7] OpenModelica. [Online]. Available: <https://openmodelica.org/>
- [8] Modelica By Example. [Online]. Available: <https://mbe.modelica.university/>
- [9] E. Branlard, *Wind Turbine Aerodynamics and Vorticity-Based Methods: Fundamentals and Recent Applications*, ser. Research Topics in Wind Energy. Cham: Springer International Publishing, 2017, vol. 7. [Online]. Available: <http://link.springer.com/10.1007/978-3-319-55164-7>
- [10] C. T. Silva and M. V. Donadon, “Unsteady blade element-momentum method including returning wake effects,” *Journal of Aerospace Technology and Management*, vol. 5, no. 1, pp. 27–42, 2013.
- [11] T. Hogan and K. Bubbar, “Augmenting the ModelicaTM Ocean Engineering Toolbox to Support Multi-body Dynamic Simulations,” in *Proceedings of the European Wave and Tidal Energy Conference*, vol. 16, 2025.
- [12] R. Peraro, “Development of an Unsteady BEM Code for Wind Turbine Applications.”
- [13] T. Hogan and K. Bubbar, “Ocean Wave Energy Converter Mid-Fidelity Numerical Simulation Tools: A Review,” in *Proceedings of the European Wave and Tidal Energy Conference*, vol. 16, 2025.

- [14] R. E. Murray, R. Thresher, and J. Jonkman, “Added-mass effects on a horizontal-axis tidal turbine using FAST v8,” *Renewable Energy*, vol. 126, pp. 987–1002, 2018.
- [15] L. Li, Y. Gao, Z. Yuan, S. Day, and Z. Hu, “Dynamic response and power production of a floating integrated wind, wave and tidal energy system,” *Renewable energy*, vol. 116, pp. 412–422, 2018.
- [16] Y. Zhu, C. Hu, Y. Ma, G. Xie, and Z. Yao, “An equation-based method for fully coupled analyses of floating offshore wind turbine based on Modelica,” *Energy Conversion and Management*, vol. 277, p. 116653, 2023.
- [17] Q. Tan, C. Huang, J. Chen, C. Hu, and Y. Ma, “A coupling framework between Modelica-AeroDyn-MoorDyn for fully coupled dynamic responses of floating offshore wind turbine with wave energy converters,” *Ocean Engineering*, vol. 341, p. 122667, 2025.